BULUT BILISIM DERSI — DONEM ODEVI

Weekly Report Portal

Mikro Hizmet Mimarisi ile Kurumsal Haftalik Raporlama Sistemi

Yasin Emin Esen

Ogrenci No: Y245012031

Nisan 2026

Icindekiler

1. Giris
- 1.1 Mikro Hizmet Mimarisinin Tanimi
- 1.2 Monolitik Yapiyla Karsilastirma
- 1.3 Bulut Bilisimle Iliskisi

2. Mimari Tasarim
- 2.1 Hizmetlerin Aciklamasi
- 2.2 Servisler Arasi Iletisim
- 2.3 Mimari Diyagram

3. Uygulama Detaylari
- 3.1 Dockerfile Aciklamalari
- 3.2 Docker Compose Aciklamasi
- 3.3 Yapay Zeka Araclarinin Kullanimi
- 3.4 Karsilasilan Zorluklar ve Cozumler

4. Olceklendirme Tartismasi

5. Sonuc

6. Kaynakca

1. Giris

1.1 Mikro Hizmet Mimarisinin Tanimi

Mikro hizmet mimarisi (microservice architecture), bir uygulamanın tek bir monolitik yapı yerine, birbirinden bağımsız çalışabilen ve her biri belirli bir iş işlevine odaklanan küçük servisler topluluğu olarak tasarlandığı bir yazılım geliştirme yaklaşımıdır. Her mikro hizmet kendi sürecinde çalışır, bağımsız olarak dağıtılabilir ve hafif iletişim mekanizmaları —genellikle HTTP/REST API'leri— aracılığıyla diğer servislerle haberleşir. Bu yaklaşım 2014 yılında Martin Fowler ve James Lewis tarafından sistematik biçimde tanımlanmış ve modern bulut tabanlı sistemlerin omurgası haline gelmiştir (Fowler & Lewis, 2014).

Mikro hizmet mimarisinin temel ilkeleri şunlardır: her servis tek bir sorumluluk alanına odaklanmalıdır (Single Responsibility Principle); servisler birbirinden gevşek bağlı (loosely coupled) ve yüksek düzeyde uyumlu (highly cohesive) olmalıdır; her servis kendi veri deposunu yönetmeli ve bağımsız olarak deploy edilebilmelidir.

1.2 Monolitik Yapiyla Karsilastirma

Monolitik mimari, tüm uygulama bileşenleri tek bir kod tabanında ve tek bir deploy edilebilir birimde bir araya getirilen geleneksel yaklaşımdır. Küçük ekipler için başlangıçta avantajlı olsa da sistem büyüdükçe ciddi sorunlara yol açar: tek bir modülde oluşan hata tüm sistemi çökertebilir; ölçeklendirme yalnızca bütünü kapsayacak şekilde yapılmak zorunda kalınır; farklı teknoloji yığınlarının benimsenmesi güçleşir.

Kriter	Monolitik Mimari	Mikro Hizmet Mimarisi
Dağıtım (Deploy)	Tüm uygulama birlikte	Her servis bağımsız
Ölçeklendirme	Tüm sistem ölçeklenir	Yalnızca ilgili servis
Hata İzolasyonu	Bir hata tümünü etkiler	Servis bazında izole
Teknoloji Seçimi	Tek teknoloji yığını	Her servis farklı teknoloji
Geliştirme Hızı	Küçük ekiplerde hızlı	Büyük ekiplerde paralel
Karmaşıklık	Düşük (başlangıçta)	Orta-yüksek (orchestration)

Tablo 1: Monolitik ve Mikro Hizmet Mimarisi Karşılaştırması

1.3 Bulut Bilisimle Iliskisi

Bulut bilişim, mikro hizmet mimarisinin en doğal yaşam ortamıdır. Ölçeklenebilirlik, yüksek erişilebilirlik ve esnek fiyatlandırma gibi bulut avantajları, yalnızca servis tabanlı mimari benimsendiğinde tam anlamıyla kullanılabilir. Konteyner teknolojileri (Docker) ve orkestrasyon platformları (Kubernetes, AWS ECS) bu iki paradigmayı birleştiren köprüdür. NIST'in bulut bilişim tanımı kapsamındaki 'on-demand self-service' ve 'rapid elasticity' özellikleri, mikro hizmetler sayesinde yazılım katmanında da gerçekleştirilebilir hale gelir (Mell & Grance, 2011).

Bu proje, Docker konteynerleri ve Docker Compose orkestrasyon aracıyla bulut-hazır (cloud-ready) bir uygulama geliştirmeyi hedeflemiştir. İlerleyen aşamalarda Kubernetes manifest dosyaları ile gerçek bir bulut ortamına (AWS/Azure/GCP) doğrudan deploy edilebilir hale getirilmiştir.

2. Mimari Tasarim

2.1 Hizmetlerin Aciklamasi

Weekly Report Portal, birbirinden bağımsız üç mikro hizmetten oluşmaktadır. Her hizmet farklı bir sorumluluğa sahip olup ayrı bir Docker konteyneri içinde çalışır.

Mikro Hizmet	Teknoloji	Sorumluluk	Port
Frontend (API Gateway)	Nginx + React (Vite SPA)	Statik dosya servisi, reverse proxy, SPA yönlendirme	3000:8080
Backend API	Node.js 20 + Express	İş mantığı, kimlik doğrulama, CRUD, PDF üretimi	5001:5000
Database	PostgreSQL 16 (Alpine)	Veri kalıcılığı, ilişkisel veri yönetimi, indeksleme	5432

Tablo 2: Mikro Hizmetlerin Özeti

Frontend servisi aynı zamanda bir API Gateway rolü üstlenir: Nginx konfigürasyonu ile gelen istekleri analiz eder ve /api/* ile /uploads/* yollarını backend servisine yönlendirir (reverse proxy). Bu sayede istemci yalnızca tek bir giriş noktası (port 3000) üzerinden tüm sisteme erişir (Newman, 2015).

2.2 Servisler Arasi Iletisim

Servisler birbirleriyle Docker'ın dahili ağı (bridge network: weekly-network) üzerinden haberleşir. Docker'ın gömülü DNS servisi sayesinde servisler birbirlerine IP adresi yerine konteyner adıyla erişebilir (örneğin http://backend:5000, postgres:5432). Kimlik doğrulama için JWT (JSON Web Token) kullanılmaktadır; her API isteğinde 'Authorization: Bearer <token>' başlığı gönderilir.

Kaynak -> Hedef	Protokol	Port (İç)	Detay
Kullanıcı -> Frontend	HTTP	8080	Browser'dan Nginx'e
Frontend -> Backend	HTTP/REST + JSON	5000	Nginx reverse proxy; JWT başlık
Backend -> Database	TCP (PostgreSQL)	5432	pg driver; pool min=2/max=10

Tablo 3: Servisler Arası İletişim Protokolleri

2.3 Mimari Diyagram

Aşağıdaki şekil, sistemin üç katmanlı mikro hizmet mimarisini ve kullanıcıdan veritabanına uzanan istek akışını göstermektedir:

```
[ KULLANICI / BROWSER ]
|  HTTP :3000
+-----------------------------------------------------+
|  FRONTEND  (Nginx + React SPA)  :8080               |
|  API Gateway -- /api/* -> backend:5000              |
|  /uploads/* -> backend:5000                         |
|  /* -> index.html (SPA)                             |
+-----------------------------------------------------+
|  REST/JSON  (Authorization: Bearer JWT)             |
+-----------------------------------------------------+
|  BACKEND API  (Node.js + Express)  :5000            |
```

```
|  Auth . Reports CRUD . PDF Export (Puppeteer)     |
|  GET /healthz   GET /readyz                       |
+---------------------------------------------------+
|  SQL over TCP (pg driver, connection pool)        |
+---------------------------------------------------+
|  DATABASE  (PostgreSQL 16)  :5432                 |
|  teams . users . reports . templates              |
|  Volume: postgres-data (kalici depolama)          |
+---------------------------------------------------+
```

Sekil 1: Uc Katmanli Mikro Hizmet Mimarisi ve Istek Akisi

Kaynak Türü	Dosya	Açıklama
Namespace	namespace.yaml	weekly-report izolasyonu
Secret	postgres-secret.yaml	DB kimlik bilgileri + JWT secret
PersistentVolumeClaim	postgres-pvc.yaml	PostgreSQL veri kalıcılığı (1Gi)
Deployment	backend-deployment.yaml	Backend (2 replica, RollingUpdate)
Deployment	frontend-deployment.yaml	Frontend (2 replica)
Service	frontend-service.yaml	NodePort :30080
Ingress	ingress.yaml	URL tabanlı yönlendirme

Tablo 4: Kubernetes Manifest Dosyaları

3. Uygulama Detaylari

3.1 Dockerfile Aciklamalari

Frontend — Dockerfile.client (Multi-Stage Build)

Frontend için çok aşamalı (multi-stage) build stratejisi benimsenmiştir. Bu yaklaşımın temel amacı, son image boyutunu minimize etmektir: build aşamasında kullanılan Node.js runtime, npm paket yöneticisi ve kaynak kodlar final image'a dahil edilmez; yalnızca Vite tarafından üretilen statik dosyalar Nginx image'ına kopyalanır.

```
FROM node:20-alpine AS build
WORKDIR /app
COPY client/package*.json ./
RUN npm ci                    # Tekrarlanabilir bagimlilik kurulumu
COPY client/ ./
RUN npm run build             # Vite -> production build (/app/dist)

FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 8080
```

Asama	Base Image	Yaklaşık Boyut	Final Image'a Dahil?
Build	node:20-alpine	~180 MB	Hayır
Production	nginx:alpine	~7 MB	Evet (final image)

Tablo 5: Multi-Stage Build Aşamaları ve Image Boyutları

Backend — Dockerfile.server

Backend image'ı Node.js 20 Alpine tabanlıdır. PDF dışa aktarma özelliği Puppeteer (headless Chromium) gerektirdiğinden, gerekli sistem kütüphaneleri Alpine paket yöneticisiyle (apk) kurulur. npm ci --omit=dev komutu ile geliştirme bağımlılıkları (jest, nodemon vb.) image'a dahil edilmez.

```
FROM node:20-alpine
RUN apk add --no-cache chromium nss freetype harfbuzz \
           ca-certificates ttf-freefont
ENV PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium-browser
WORKDIR /app
COPY server/package*.json ./server/
RUN cd server && npm ci --omit=dev   # Sadece production bagimliliklari
COPY server/ ./server/
EXPOSE 5000
CMD ["node", "index.js"]
```

3.2 Docker Compose Aciklamasi

docker-compose.yml dosyası, üç mikro hizmetin birlikte yönetilmesini sağlar. En önemli tasarım kararı, sağlık kontrolü (health check) tabanlı başlama zinciridir: PostgreSQL pg_isready testini geçmeden Backend başlamaz; Backend /healthz endpoint'inden 200 dönmeden Frontend çalışma başlamaz. Bu zincirleme yapı, servis hazırlığını garanti eder ve race condition kaynaklı hataları önler.

```
services:
  postgres:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL","pg_isready -U postgres -d weekly_report"]
      interval: 5s
      retries: 10
```

```
    volumes:
      - postgres-data:/var/lib/postgresql/data

  backend:
    build: { dockerfile: Dockerfile.server }
    depends_on:
      postgres: { condition: service_healthy }
    environment:
      - DB_ADAPTER=postgresql
      - DB_HOST=postgres         # Docker DNS ile servis adi

  frontend:
    build: { dockerfile: Dockerfile.client }
    depends_on:
      backend: { condition: service_healthy }

networks:
  weekly-network:
    driver: bridge
```

Volume	Mount Noktasi	Amaç
postgres-data	/var/lib/postgresql/data	Veritabanı kalıcılığı
uploads-data	/app/server/uploads	Kullanıcı dosya yüklemeleri
init.sql (bind)	docker-entrypoint-initdb.d/	İlk çalıştırmada tablo oluşturma

Tablo 6: Docker Volume Yapılandırması

3.3 Yapay Zeka Araclarinin Kullanimi

Bu projenin geliştirilme sürecinde, mimari tasarım ve kod yazımı aşamalarında çeşitli yapay zeka (YZ) araçlarından yararlanılmıştır. Söz konusu araçlar doğrudan kod üretmek yerine bir 'akıllı asistan' rolünde kullanılmış; üretilen her çıktı eleştirel gözle incelenerek projenin özgün gereksinimlerine göre uyarlanmıştır.

Araç	Kullanım Amacı	Katkı Alanı
Claude (Anthropic)	Mimari karar desteği, kod gözden geçirme	Docker Compose yapısı, Nginx konfigürasyonu, API tasarımı
Google Gemini	Alternatif yaklaşım analizi, dokümantasyon desteği	PostgreSQL şema optimizasyonu, Kubernetes manifest yapısı

Tablo 7: Geliştirme Sürecinde Kullanılan YZ Araçları

Yapay zeka araçlarının en somut katkıları şunlardır: Docker multi-stage build stratejisinin belirlenmesi, Nginx reverse proxy konfigürasyonunun hazırlanması, sağlık kontrolü tabanlı servis başlama zincirinin tasarlanması ve Kubernetes manifest dosyalarının oluşturulması. Tüm öneriler projenin özgün gereksinimleriyle karşılaştırılmış ve gerekli durumlarda el ile düzeltilmiştir. Veritabanı şema tasarımı, kimlik doğrulama akışı, PDF üretim mekanizması ve frontend bileşenleri tamamen geliştirici tarafından yazılmış ve entegre edilmiştir.

3.4 Karsilasilan Zorluklar ve Cozumler

1. **Puppeteer / Chromium Bağımlılığı:** Headless Chrome, Alpine Linux üzerinde doğrudan çalışmıyor; eksik sistem kütüphaneleri (nss, freetype, harfbuzz) hata veriyordu. Çözüm: apk add ile gerekli

kütüphaneler Dockerfile'a eklendi ve PUPPETEER_EXECUTABLE_PATH ortam değişkeniyle sistem Chromium'una yönlendirildi.

2. **Servis Başlama Sırası (Race Condition):** Backend, PostgreSQL hazır olmadan başladığında bağlanma hatası fırlatıyordu. Çözüm: Docker Compose 'condition: service_healthy' özelliği ve pg_isready sağlık kontrolü ile zincirleme başlama sırası oluşturuldu.

3. **Veritabanı Adapter Pattern:** Geliştirme ortamında PostgreSQL kurulumu gerektirmemek için JSON dosya tabanlı bir adapter yazıldı. Aynı interface'i implemente eden JsonAdapter ve PgAdapter, DB_ADAPTER ortam değişkeniyle seçilir (Gamma et al., 1994).

4. **Multi-Stage Build Optimizasyonu:** İlk Dockerfile denemeleri ~800 MB image üretiyordu. Multi-stage build ile build bağımlılıkları çıkarılarak frontend image boyutu ~20 MB'a indirildi (Docker, 2024).

5. **JWT / CORS Entegrasyonu:** Geliştirme ortamında farklı portlarda çalışan frontend ve backend arasında CORS hataları oluştu. Vite dev proxy ve production Nginx reverse proxy konfigürasyonu ile tüm istekler aynı origin üzerinden geçirilerek sorun çözüldü.

4. Olceklendirme Tartismasi

Mikro hizmet mimarisinin en kritik avantajlarından biri, bileşenlerin bağımsız olarak yatay (horizontal) ölçeklendirilebilmesidir. Monolitik bir yapıda tüm uygulama tek bir birim olarak ölçeklenirken, bu projede her servis trafik yüküne göre ayrı ayrı ölçeklendirilebilir.

Kubernetes ile Yatay Olceklendirme

Kubernetes ortamında bir servisin kapasitesini artırmak için yalnızca replica sayısını değiştirmek yeterlidir. RollingUpdate stratejisi sayesinde güncelleme işlemi sıfır kesinti (zero-downtime) ile gerçekleşir:

```
# Backend'i 5 instance'a yuksel
kubectl -n weekly-report scale deployment/backend --replicas=5

# Frontend'i 3 instance'a yuksel
kubectl -n weekly-report scale deployment/frontend --replicas=3
```

Servis	Ölçekleme Kolaylığı	Dikkat Edilecek Nokta
Frontend (Nginx+React)	Kolay — tam stateless	Load balancer arkasında çoklu instance; session bilgisi tutulmaz
Backend API (Node.js)	Kolay — büyük ölçüde stateless	JWT stateless auth; dosya yüklemeleri PVC ile paylaşılıyor olmalı
Database (PostgreSQL)	Dikkatli — stateful workload	Read replica'lar eklenebilir; yazma ölçekleme için sharding gerekir

Tablo 8: Servis Bazlı Ölçeklendirme Stratejisi

Otomatik ölçeklendirme için Kubernetes Horizontal Pod Autoscaler (HPA) kullanılabilir. HPA, CPU kullanımı veya özel metrikler eşiğini aştığında pod sayısını otomatik olarak arttırır ya da azaltır.

Servis	CPU Request	CPU Limit	Memory Request	Memory Limit
PostgreSQL	100m	500m	256Mi	512Mi
Backend	100m	500m	256Mi	512Mi
Frontend	50m	200m	64Mi	128Mi

Tablo 9: Kubernetes Konteyner Kaynak Sınırları

Metrik	Gözlemlenen Değer
Toplam başlama süresi (ilk build)	~5-10 dakika
Toplam başlama süresi (cache ile)	~15 saniye
PostgreSQL health check geçiş süresi	~6 saniye
Backend readiness suresi	~11 saniye
API yanıt süresi (GET /healthz)	< 10 ms

Metrik	Gözlemlenen Değer
Frontend ilk yükleme (gzip)	< 500 ms

Tablo 10: Docker Compose Ortamında Ölçülen Performans Değerleri

Veritabanı katmanında ölçeklendirme en zorlu kısımdır. PostgreSQL için streaming replication ile okuma yükü dağıtılabilir; yüksek yazma hacmi durumunda Citus uzantısı veya dış bir shard yönetim katmanı değerlendirilmelidir. Gelecekte Redis önbellek katmanı eklenerek sık erişilen rapor sorgularının veritabanını tekrar tekrar sorgulaması önlenebilir.

5. Sonuc

Bu proje, modern yazılım geliştirme pratiklerinin bir arada uygulandığı kapsamlı bir full-stack çalışmadır. Mikro hizmet mimarisi, Docker konteyner teknolojisi ve Kubernetes orkestrasyonunun entegrasyonu sayesinde her bileşen bağımsız olarak geliştirilebilir, test edilebilir ve ölçeklendirilebilir hale getirilmiştir.

Ogrenilenler

- Mikro hizmet geçişi yalnızca teknik değil aynı zamanda organizasyonel bir karardır; her servisin bağımsız CI/CD zinciri olmadan faydası sınırlı kalır.
- Docker multi-stage build, image boyutunu dramatik biçimde küçültüp hem depolama maliyetini hem de saldırılabilir yüzey alanını azaltır.
- Sağlık kontrolü tabanlı servis sıralaması, race condition kaynaklı başlama hatalarını engelleyen kritik bir tasarım örüntüsü (pattern) olarak öne çıkmaktadır.
- Veritabanı adapter deseni (JsonAdapter / PgAdapter), geliştirme ortamı karmaşıklığını azaltırken production geçişini sorunsuz hale getirmiştir.
- Yapay zeka araçları (Claude, Gemini) mimari kararları hızlandırmıştır; ancak üretilen her çıktının projeye özgü gereksinimler çerçevesinde eleştirel biçimde değerlendirilmesi zorunludur.

Konteyner Teknolojilerinin Bulut Bilisimdeki Onemi

Konteyner teknolojileri, bulut bilişimin 'önce yaz, her yerde çalıştır' vaadini gerçek anlamda hayata geçirmiştir. Docker ile paketlenen bir uygulama geliştirici dizüstü bilgisayarından üretim sunucusuna aynı davranışı sergiler. Kubernetes bu ortam bağımsızlığını orkestrasyon katmanına taşıyarak otomatik iyileşme (self-healing), bildirimsel altyapı yönetimi ve servis keşfi sağlar. Burns et al. (2016) tarafından aktarılan bu ilkeler, büyük ölçekli sistemlerin güvenilir biçimde işletilmesinin temelini oluşturmaktadır.

Gelecekte bu projeye eklenebilecek iyileştirmeler: Prometheus + Grafana ile metrik izleme, ELK Stack ile merkezi log yönetimi, GitHub Actions ile otomatik CI/CD pipeline ve Redis önbellek katmanı. Bu eklemeler uygulamayı enterprise düzey bir bulut platformuna taşımak için yeterli altyapıyı oluşturacaktır.

6. Kaynakca

[1] Fowler, M., & Lewis, J. (2014). Microservices: A Definition of This New Architectural Term. martinfowler.com. https://martinfowler.com/articles/microservices.html

[2] Newman, S. (2015). Building Microservices: Designing Fine-Grained Systems. O'Reilly Media. ISBN: 978-1491950357.

[3] Docker Inc. (2024). Best practices for writing Dockerfiles. Docker Documentation. https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

[4] Kubernetes Authors. (2024). Concepts: Workloads, Services, Networking. Kubernetes Documentation. https://kubernetes.io/docs/concepts/

[5] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. ACM Queue, 14(1), 70-93. https://doi.org/10.1145/2898442.2898444

[6] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. ISBN: 978-0201633610.

[7] Mell, P., & Grance, T. (2011). The NIST Definition of Cloud Computing (SP 800-145). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-145